Lecture Notes: OpenMP Performance — A Case Study with Monte Carlo π

HPC 1 — Università di Trieste, A.Y. 2025/2026

Luca Tornatore

What this lecture is about

We have a set of six C codes. They all do the same thing — estimate π with a Monte Carlo method — but they differ in how they handle the parallel accumulation of results. The algorithm is trivial on purpose; nobody cares about the algorithm. What we care about is performance, and specifically *why* one synchronisation strategy can be orders of magnitude faster than another. The reasons, as usual, live in the hardware.

Here are the codes, in the order we'll look at them:

File	Strategy	Expected relative performance
`pi_openmp.uncorrect_random.c`	Thread-unsafe vs. thread-safe PRNG	(correctness, not speed)
`pi_openmp.v0.c`	`critical` on every iteration	Worst
`pi_openmp.v0b.c`	`critical` only on hits (≈ π/4 of iterations)	Second worst
`pi_openmp.v1.c`	`atomic` on every iteration	Better
`pi_openmp.v2.c`	Local accumulator + final `atomic`	Best
`pi_openmp.use_reduction.c`	`omp for` with `reduction(+:Nvalid)`	Same as v2

Every code prints a per-thread wall-clock time, so you can directly see whether the threads are balanced or not.

We will also compare results from two machines — a 4-socket Skylake-SP node and a 2-socket Broadwell-EP node — and we'll see that the *topology* of the machine amplifies the synchronisation overhead in a way you wouldn't necessarily expect. Same clock speed, comparable core count, yet very different performance for the bad versions; v2 is unaffected.

Both platforms are roughly 10 years old at this point.
Exercise: run the same test on a Leonardo@CINECA node.

1. The algorithm: Monte Carlo estimation of π

Consider the unit square [0,1]×[0,1] and the quarter-circle of radius 1 inscribed in it.
The area of the quarter-circle is π/4. If you throw N random points uniformly in the square, the fraction that lands inside the quarter-circle converges to π/4 as N → ∞.

So: generate pairs (x, y) with x, y ∈ [0,1), count how many satisfy $x^2 + y^2 < 1$, call that Nvalid. Then

$$\pi \approx 4 \cdot \frac{N_{\text{valid}}}{N_{\text{total}}}$$

where N_{total} is the total number of points thrown across all threads.

The statistical error goes as 1/√N, which is outrageous for a numerical method — you would never actually compute π this way. But the algorithm is embarrassingly parallel: each random throw is independent, which makes it a clean vehicle for studying synchronisation overhead.

In all the codes, every thread generates N points (not N/nthreads), and the denominator is correspondingly N × nthreads. This means the total work scales linearly with the thread count. If the parallelisation were perfect (zero overhead), the wallclock time
would stay constant. Any deviation from constant wallclock is synchronisation overhead, and that's what we're after.

This is a ***weak scaling*** test: the problem size grows with the number of threads,and the metric of success is constant execution time.

---

2. The two test platforms

Before looking at the codes, let's describe the two machines. They have the same base clock (2.30 GHz) but differ in pretty much everything else that matters for synchronisation.

Platform A: 2-socket Broadwell-EP (E5-2697 v4)

```
┌──────────────────────────────┐       QPI        ┌──────────────────────────────┐
│         Socket 0             │◄────────────────►│         Socket 1             │
│                              │   (9.6 GT/s)     │                              │
│   18 cores, 36 threads (HT)  │    1 hop         │   18 cores, 36 threads (HT)  │
│   45 MB L3 (inclusive)       │                  │   45 MB L3 (inclusive)       │
│   4 × DDR4-2400 channels     │                  │   4 × DDR4-2400 channels     │
│                              │                  │                              │
│   Internal: dual ring bus    │                  │   Internal: dual ring bus    │
│   (9 cores per ring, ~5 cyc  │                  │   (9 cores per ring, ~5 cyc  │
│    penalty for ring-to-ring) │                  │    penalty for ring-to-ring) │
└──────────────────────────────┘                  └──────────────────────────────┘
         Total: 36 cores / 72 threads, 2 sockets
         Inter-socket: 1 direct QPI link (every socket pair: 1 hop)
```

Platform B: 4-socket Skylake-SP (Gold 5118)

```
┌──────────────────┐     UPI      ┌──────────────────┐
│    Socket 0      │◄────────────►│    Socket 1      │
│  12 cores, no HT │              │  12 cores, no HT │
│  16.5 MB L3      │              │  16.5 MB L3      │
│  (non-inclusive) │              │  (non-inclusive) │
│  6 × DDR4-2400 ch│              │  6 × DDR4-2400 ch│
│  Internal: mesh  │              │  Internal: mesh  │
└────────┬─────────┘              └────────┬─────────┘
         │ UPI                             │ UPI
         │                                 │
┌────────┴─────────┐              ┌────────┴─────────┐
│    Socket 3      │◄────────────►│    Socket 2      │
│  12 cores, no HT │     UPI      │  12 cores, no HT │
│  16.5 MB L3      │              │  16.5 MB L3      │
│  (non-inclusive) │              │  (non-inclusive) │
│  6 × DDR4-2400 ch│              │  6 × DDR4-2400 ch│
│  Internal: mesh  │              │  Internal: mesh  │
└──────────────────┘              └──────────────────┘

        Total: 48 cores / 48 threads (HT off), 4 sockets
        Inter-socket: ring of 2 UPI links per socket
        Socket 0 ↔ Socket 2: 2 hops (via Socket 1 or Socket 3)
```

The key difference: on Platform A, any core can reach any other core's cache in at most **1 inter-socket hop** (the direct QPI link). On Platform B, the Gold 5118 has only 2 UPI links, so in a 4-socket configuration the sockets form a *ring* — not fully connected. To get from socket 0 to socket 2 you must pass through socket 1 or 3: **2 hops**.
That roughly doubles the worst-case coherence latency.

Since every `critical` or `atomic` operation on a shared variable forces the cache line containing that variable to travel between cores, the topology is quite important. More hops → more time per synchronisation event → and in the bad versions, we have *billions* of these events. In v2 there's essentially no inter-core traffic during the hot loop, so the topology becomes invisible.

---

3. Step zero: the random number generator problem

Before we talk about synchronisation, there is a subtler problem that can destroy the *correctness* of the result: thread safety of the PRNG.

`pi_openmp.uncorrect_random.c` illustrates this. It compiles in two modes, controlled by a preprocessor flag.

The unsafe version (default, without `-DUSE_CORRECT_RND`):

```
srand48(SEED*(myid+1));
#define RND_VAL drand48()
```

The safe version (with `-DUSE_CORRECT_RND`):

```
int unsigned short myseeds[3] = {SEED+(myid), SEED^(myid*3+1), SEED&(myid*4+2)};
seed48( myseeds );
#define RND_VAL erand48(myseeds)
```

Why `drand48()` is broken with threads

`drand48()` keeps its state in a single global 48-bit buffer — a static variable inside the C library. When multiple threads call it concurrently, they read-modify-write that buffer without any locking. Two consequences:

1. **Data race**: the interleaving corrupts the linear congruential sequence. It's no longer the deterministic sequence the generator is supposed to produce.
2. **Statistical bias**: the corrupted sequence can have correlations, reduced period, or other defects. With enough points, you *will* see the π estimate converge to the wrong value.

The fix: `erand48()` with per-thread state

`erand48()` is the reentrant cousin of `drand48()`. It takes a pointer to a caller-supplied 3-element `unsigned short` array that holds the state. Each thread keeps its own seed array on its own stack — no shared mutable state at all.

The seeding is intentionally minimal: we derive per-thread seeds from a base SEED and the thread id with simple bit operations (addition, XOR, AND). For a real simulation you'd want a proper parallel PRNG (Philox, Threefry, L'Écuyer's MRG32k3a), but for estimating π this is fine.

This is very general. Any library function that stores mutable state internally is a trap in a multi-threaded context. `strtok()`, `ctime()`, `gethostbyname()` — POSIX marks them all as "not required to be thread-safe." Always check for reentrant variants (the `_r` suffix is the POSIX convention; C11 uses `_s` suffixes).

---

4. Version 0: `critical` on every iteration

Here is the inner loop of `pi_openmp.v0.c`:

```
#pragma omp parallel
{
    // ... seed setup ...

    for( long long unsigned int i = 0; i < N; i++)
    {
        double x = erand48(myseeds);
        double y = erand48(myseeds);

       #pragma omp critical
        Nvalid += ( (x*x + y*y) < 1.0 );
    }
}
```

Every single iteration, the thread enters a critical section to update the shared accumulator, regardless of whether the point fell inside or outside the quarter-circle.

What happens at the hardware level

A `critical` in OpenMP is a mutex:
Thread enters → acquires lock → all other threads block (spin or sleep) → thread exits → lock released → memory flush of shared variables.

The cost is large, and stacks up from several sources:

Lock acquisition. Even uncontended, a mutex acquire-release takes 20–50 ns. Comparable to a DRAM access. And it happens *every iteration*.

Serialisation. With T threads hammering the same critical section, only one gets in at a time. The useful work per iteration is maybe 5–10 ns (two `erand48` calls + comparison). So T−1 threads are idle most of the time. The code is effectively serial, but with the extra overhead of lock management on top.

Cache-line bouncing. Nvalid is 8 bytes sitting in a 64-byte cache line. Each write pushes it to Modified state in the writer's L1 and invalidates it everywhere else. The next thread must fetch the updated line via a cache-to-cache transfer (30–70 cycles on-die, 100–200 cycles cross-socket). This is the MESI protocol in action — correct, but expensive when you do it a billion times.

Memory flush. OpenMP mandates a flush at entry and exit of a critical section. On x86 (TSO memory model) this is cheaper than on ARM, but the coherence traffic is still there.

What the data shows

On the 4-socket platform (`scalability.v0.png`), per-thread time grows roughly linearly from a fraction of a second to over 80 seconds at 48 threads. That's not "failing to speed up" — each thread is individually getting *slower* as you add threads, because it spends more time waiting for the lock.

On the normalised plot (`scalability_normalized_all.png`, 4-socket), v0 hits about **2600×** at 48 threads. Each thread's loop takes 2600 times longer than the single-thread case.

On the 2-socket platform, the same version at 36 threads sits at roughly **600×**. Same algorithm, same clock speed, but the penalty is ~4× less severe. We'll come back to why in Section 10.

One curious detail: the standard-deviation bands on v0 are very tight. All threads are equally slow. The mutex serialises them so completely that everyone waits about the same amount. There's a perverse fairness in total serialisation.

Rough quantitative estimate

Say the useful work per iteration is ~10 ns and the critical-section overhead (lock + cache-line transfer + unlock) is ~100 ns. With T threads each doing N iterations, but only one thread in the critical section at a time:

$$t_{\text{wall}} \approx N \cdot T \cdot 100\,\text{ns}$$

while without the critical section:

$$t_{\text{serial}} \approx N \cdot 10\,\text{ns}$$

The parallel version is slower than serial by a factor of ~10T. Adding threads makes it worse.

---

5. Version 0b: `critical` only on hits — marginally better

`pi_openmp.v0b.c` guards the critical section with an `if`:

```
for( long long unsigned int i = 0; i < N; i++)
{
    double x = erand48(myseeds);
    double y = erand48(myseeds);

    if ( (x*x + y*y) < 1.0 )
    {
       #pragma omp critical
        Nvalid++;
    }
}
```

The critical section is entered only when the point is inside the quarter-circle — about 78.5% of the time ($\pi/4 \approx 0.785$).

Why this barely helps

You save at most a factor of $1/0.785 \approx 1.27$ in the number of lock acquisitions. Since the lock dominates the runtime, that's maybe 20–25% faster. The problem is the same: you're still entering a mutex-protected region on the vast majority of iterations.

Notice also the difference in what's inside the critical section: v0 does `Nvalid += (x*x + y*y < 1.0)` (always enters, adds 0 or 1); v0b does `Nvalid++` (enters only when the condition is true). So v0b genuinely enters fewer times. But since $\pi/4$ is close to 1, the saving is modest.

What the data shows

On the 4-socket, v0b reaches ~**2000×** at 48 threads — better than v0's 2600×, but same ballpark of disaster. On the 2-socket, v0 and v0b are nearly indistinguishable at 36 threads, both around 600×.

---

6. Version 1: `atomic` on every iteration

```
for( long long unsigned int i = 0; i < N; i++)
{
    double x = erand48(myseeds);
    double y = erand48(myseeds);

   #pragma omp atomic update
    Nvalid += ( (x*x + y*y) < 1.0 );
}
```

How `atomic` differs from `critical`

Let me stress that once more.

A `critical` protects a *code region*. Software mutex: lock → execute whatever code → unlock → flush. Threads queue up.

An `atomic` protects a *memory location*. On x86, for an integer addition, the compiler emits a `LOCK ADD` or `LOCK XADD` — a single hardware instruction with the LOCK prefix. The prefix holds the cache-line lock for the duration of the read-modify-write. No software mutex, no thread queue, no kernel.

There's still a cost — the LOCK prefix forces the line to Exclusive/Modified state and stalls other cores trying to access it — but it's much cheaper. A contended LOCK ADD: ~30–80 cycles. A full mutex round-trip: 100+ cycles with pipeline flushes on top.

`atomic` and `critical` do NOT protect each other

Worth repeating: `atomic` protects memory, `critical` protects code. They're independent mechanisms.

A thread doing an `atomic` on Nvalid will *not* block another thread entering a `critical` that touches the same Nvalid. If you mix them on the same variable, however, you risk a data race.

What the data shows

Big improvement — roughly an order of magnitude over v0. On the 4-socket, v1 reaches ~**160×** at 48 threads (vs. 2600× for v0). On the 2-socket, ~**100–150×** at 36 threads.

Look carefully at the 2-socket v1 plot (`scalability.v1.png`). There are striking periodic spikes — sharp upward jumps at roughly regular intervals. These are a fingerprint of the Broadwell-EP dual-ring topology: when thread placement forces traffic across the ring-to-ring bridge, or across the socket boundary, latency jumps. The 4-socket v1 plot is smoother — the mesh distributes traffic more evenly, even though its average latency is higher.

Still: v1 is *still* serialised at the level of the cache line. Every iteration pays a contended atomic and the scalability is still poor.

---

7. Version 2: local accumulators — how it should be done

```
#pragma omp parallel
{
    // ... seed setup ...

    long long unsigned int local_valid_points = 0;
    for( long long unsigned int i = 0; i < N; i++)
    {
        double x = erand48(myseeds);
        double y = erand48(myseeds);

        local_valid_points += ( (x*x + y*y) < 1.0 );
    }
```

```
    #pragma omp atomic update
     Nvalid += local_valid_points;
 }
```

Simple change, enormous effect. Each thread accumulates in a *private* local variable. Only at the very end — once, after all N iterations — does it touch the shared Nvalid.

Why this works

`local_valid_points` lives on the thread's stack. Private. No other thread ever reads or writes it. From the coherence point of view, the cache line holding it is Exclusive for the entire loop. No coherence traffic, no bus locks, no stalls. The increment is a plain `ADD` — about as cheap as a CPU instruction gets.

The single atomic update at the end: one LOCK ADD, per thread, once. With $N = 10^6$ and T = 8, you go from 8 million contended atomics (v1) to 8. The overhead drops by a factor of N.

What the data shows

On the normalised plot, v2 is essentially invisible — it sits at ~1.0 while everything else explodes. Per-thread time on both platforms: flat around 0.015–0.020 s, regardless of thread count (within physical cores).

The scatter on the 4-socket v2 plot is noticeable — maybe ±20% — but that's system noise (OS jitter, NUMA placement, turbo variations). Compare with v0's 2600× degradation.

About the efficiency plot for v2

The efficiency plot on the 4-socket shows T_1/T_N *above* 1.0 — around 1.1–1.3. "Super-linear weak scaling" sounds exciting, but it's just **turbo boost**. With 1 thread, the sole active core runs at 3.2 GHz (Gold 5118 turbo). With 48 threads, everybody runs at the base 2.3 GHz. So T_1 was measured at a faster clock than T_N, and the ratio comes out above 1.

The 2-socket v2 efficiency and hyperthreading:

The 2-socket v2 efficiency tells a different story: it declines to ~0.83–0.85 up to 36 threads, then it drops linearly to ~0.42 at 72 threads. The knee at 36 is the HT boundary — beyond that, two hyperthreads share each core's execution resources. The HT factor works out to ~0.42/0.83 ≈ 0.5, i.e. each hyperthread gets about half the throughput of a dedicated core, which is typical for compute-bound work.

A punchline on Superlinearity

"Super-linear" in weak scaling →probably a measurement bug.
"Super-linear" in strong scaling → can be real (working set drops into a faster cache level).
Different mechanisms, different interpretation.

---

8. The `reduction` clause — letting OpenMP handle it

`pi_openmp.use_reduction.c` uses `omp for` with a `reduction` clause:

```
#pragma omp parallel
{
    // ... seed setup ...

   #pragma omp for reduction(+:Nvalid)
    for( long long unsigned int i = 0; i < N; i++)
    {
        double x = erand48( myseeds );
        double y = erand48( myseeds );

        Nvalid += ( (x*x + y*y) < 1.0 );
    }
}
```

What `reduction(+:Nvalid)` does

The runtime creates a private copy of Nvalid per thread (initialised to 0), then combines all copies at the end of the worksharing loop. This is what we did by hand in v2: local accumulator per thread, combine once.

Structural difference from v2

In v2 every thread runs the full loop `for i = 0 to N`. Total points: N × nthreads.

With `omp for`, the loop gets *distributed*: each thread gets roughly N/T iterations.
Total points: N.
This form is better suited for strong scaling.

Performance

Same as v2. The `reduction` clause is syntactic sugar for the local-accumulator pattern. GCC and LLVM generate the same code.

---

9. Hierarchy of synchronisation costs

Version	Sync per iteration	Cost per sync	Total sync cost	Scalability
v0	1 `critical` per iter	~200+ cycles (mutex + flush)	N × T × 200+ cycles	Anti-scales (worse with T)
v0b	~0.785 `critical` per iter	same	0.785 × N × T × 200+	Anti-scales
v1	1 `atomic` per iter	~30–80 cycles (LOCK ADD)	N × T × 30–80 cycles	Poor (contention grows with T)
v2	1 `atomic` total per thread	~30–80 cycles	T × 30–80 cycles	Near-perfect

Version	Sync per iteration	Cost per sync	Total sync cost	Scalability
reduction	(same as v2)	(same as v2)	T × 30–80 cycles	Near-perfect

The point is not "atomic beats critical" — though it usually does for simple operations.
The actual point is: **don't synchronise inside a tight loop if you can avoid it; accumulate locally, combine once.**
This is the most common performance pattern in shared-memory programming.

---

10. Same code, two platforms: why topology matters

Comparison at 36 threads, from the normalised plots:

Version	4-socket Gold 5118 @ 36 thr	2-socket E5-2697 v4 @ 36 thr
v0	~1500×	~600×
v0b	~1300×	~600×
v1	~100×	~60×
v2	~1×	~1×

Contention-heavy versions: 2–3× worse on the 4-socket. Synchronisation-free version: identical.

Why

Every `critical` or `atomic` on a shared variable forces a cache-line transfer. The cost depends on how far the line has to travel.

On the 2-socket Broadwell-EP, any inter-socket transfer is **1 hop** — one other socket, directly connected.

On the 4-socket Gold 5118, the ring topology means diagonally opposite sockets are **2 hops** apart. Socket 0 → Socket 2 passes through Socket 1 or 3. Roughly double the coherence latency for half the socket pairs, and the intermediate socket's UPI links carry transit traffic on top of their own — creating congestion.

Also, 4 sockets = more participants in the coherence protocol: every write must potentially invalidate copies in 3 other sockets instead of 1.

v2 generates essentially zero inter-core traffic during the loop, so none of this matters.

The general point: the cost of synchronisation is not a fixed number you can look up. It depends on the machine. Same code, same compiler, same clock — different topology, different performance. The difference is entirely in the synchronisation.

There are additional factors that add-up, but they are too technical for this note.

---

11. Hardware details: cache-line bouncing and false sharing

Some connection to the architecture lectures.

Cache-line bouncing (MESI)

In v0 and v1, Nvalid sits in one cache line. Every write pushes it to M(odified) in the writer's L1 and invalidates it everywhere else. The next thread must fetch the updated line — via cache-to-cache transfer on-die (30–50 cycles) or cross-socket (100–200 cycles, double that for a 2-hop path).

In v2, the line with Nvalid is touched once per thread at the end. Each thread's local counter stays Exclusive the whole time. No bouncing.

False sharing — a trap this code avoids

In v2, `local_valid_points` is on the thread's stack — stacks are separated by at least a page, so no false sharing. But imagine the wrong way to do it:

```
long long unsigned int counts[nthreads];
// ...
#pragma omp parallel
{
    int tid = omp_get_thread_num();
    for (...) {
        counts[tid] += (...);
    }
}
```

With a small `nthreads`, the whole array fits in one or two cache lines. Threads write to different elements, but those elements share a line. You get bouncing even though no two threads touch the same variable.
This is **false sharing**, and it can be as bad as true sharing.
Fix: pad each element to a full cache line (64 bytes), or better, just use stack-local variables.

---

12. A note on the seeding pattern

All the "correct" codes (v0, v0b, v1, v2) seed like this:

```
long int SEED = time(NULL);
int unsigned short myseeds[3] = {(short)(SEED+(myid)),
                                 (short)(SEED^(myid*3+1)),
                                 (short)(SEED&(myid*4+2))};
seed48( myseeds );
```

`use_reduction.c` instead uses a fixed `#define SEED 918273`.

`time(NULL)` as a base seed is fine for pedagogy. For **reproducibility** (and you usually want reproducibility), use a fixed seed. The mixing with `myid` is minimal — it ensures different threads get different initial states, but doesn't guarantee statistical independence of the streams. For serious Monte Carlo work (lattice QCD, molecular dynamics, ...) use a proper
parallel PRNG.

`seed48` / `erand48` are POSIX, not C standard. They work on Linux and macOS; on Windows you'd need WSL or a different generator.

---

13. Exercises and discussion

Exercises

1. **Compile and run all versions** with increasing N (10^5, 10^6, 10^7, 10^8) and increasing thread counts (1, 2, 4, 8, 16). Record wallclock times and per-thread times.
2. **Plot weak scaling** for each version: per-thread execution time vs. thread count. v0 and v0b should anti-scale (time grows linearly with T); v1 should grow more slowly; v2 should stay flat.
3. **Implement a false-sharing version**: replace the local accumulator in v2 with a shared array `counts[nthreads]`, indexed by thread id. Measure the degradation. Then add padding (`__attribute__((aligned(64)))` or a padding struct) and measure again.
4. **Use `perf stat`** or `likwid-perfctr` to measure cache misses and bus transactions for v0 vs. v2. You should see dramatically higher L1/LLC miss rates and interconnect traffic in v0.
5. **RUN THE BENCHMARK ON A LEONARDO NODE** (different socket count, different interconnect topology).

---

14. Summary

This exercise is small, but the principle is general: **minimise synchronisation on shared state.**
OpenMP gives you `critical`, `atomic`, `reduction`, local accumulators — from maximally expensive to essentially free. The cost hierarchy comes from cache coherence, bus locking, and mutex overhead.

"Accumulate locally, reduce once" is not a trick. It's the default design for any parallel reduction. If you see an `atomic` or a `critical` inside a tight loop, treat it as a red flag: there's almost certainly a way to push the synchronisation to the boundary.

And from the platform comparison: **synchronisation costs are not portable.** Same code, same clock, different topology → very different performance for contention-heavy code. The only robust strategy is to eliminate unnecessary synchronisation altogether. Then the topology doesn't matter.

Cross-references: MESI protocol and cache coherence → Intro to Parallelism slides 81–85.
`critical`, `atomic`, `reduction` semantics → Parallel Regions slides 35–45.
False sharing and variable classification → Intro to Parallelism slide 85.

Reference plots

4-socket Intel Xeon Gold 5118 (Skylake-SP, 48 cores, HT off):
`scalability_normalized_all.png` — all four versions compared (the key plot),
`scalability.v0.png` through `scalability.v2.png` — absolute per-thread time,
`efficiency.v0.png` through `efficiency.v2.png` — T_1/T_N ratio,
`speedup.v0.png` through `speedup.v2.png` — T_N/T_1 ratio.

2-socket Intel Xeon E5-2697 v4 (Broadwell-EP, 36 cores, HT on → 72 threads):
Same set. Thread counts beyond 36 use hyperthreading; the v2 efficiency plot shows the HT knee at T = 36.